



Факултет по Математика и Информатика  
ПУ "Паисий Хилендарски"

# ОРКЕСТРАЦИЯ

Витали Атиас



Защо е нужна  
оркестрация?

| Проблем / Предизвикателство | Как изглежда без оркестрация                                                                   | Как оркестрацията решава проблема                                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Manual Deployment           | Ръчно стартиране на стотици контейнери чрез SSH → бавно, грешно и неефективно                  | Централизиран контрол: автоматично деплойване, мащабиране и управление на всички контейнери от един control plane  |
| Resource Management         | Трудно е да се реши на кой host да се пусне нов контейнер; възможно е неравномерно натоварване | Оркестраторът (напр. Kubernetes Scheduler) автоматично избира най-подходящия host според ресурсите и натоварването |
| Service Discovery           | Контейнерите имат динамични IP адреси; трудно се намират един друг                             | Автоматична service discovery чрез DNS и virtual networking (service names вместо IP адреси)                       |
| Load Balancing              | Няма централен механизъм за разпределяне на заявките между контейнери                          | Вграден load balancer, който насочва трафика към здравите и налични контейнери                                     |
| Health Monitoring           | Ръчни проверки дали контейнерите работят; при срыв – няма автоматично възстановяване           | Автоматични health checks и restart при failure (self-healing)                                                     |
| Rolling Updates             | Ръчно спиране и пускане на нови версии; риск от грешки и downtime                              | Автоматизирани rolling updates с възможност за rollback при проблем                                                |
| Auto-Scaling                | Няма автоматична реакция при промяна в натоварването                                           | Horizontal auto-scaling: оркестраторът автоматично стартира/спира контейнери според трафика и ресурсите            |
| Self-Healing                | Срыв на контейнер или host → услугата спира                                                    | Оркестраторът автоматично рестартира, пресъздава или премества контейнерите, пренасочва трафика                    |
| Configuration Management    | Различни конфигурации, секрети и файлове на стотици контейнери – трудно за управление          | Централизирано управление на конфигурации и secrets; промени без downtime                                          |
| Storage Orchestration       | Ръчно управление на обеми, липса на автоматично прикачване и backup                            | Динамично създаване и свързване на persistent storage, backup/restore, cross-node достъп                           |

# Какво представлява?

Автоматизирано координиране на множество елементи в една система – това може да са контейнери, виртуални машини, облачни ресурси, или цели workflows.

Оркестрацията = автоматизация + координация на много взаимосвързани компоненти — не само контейнери, но и цялата инфраструктура около тях.

| Характеристика               | Оркестрация на контейнери               | Оркестрация на виртуални машини        |
|------------------------------|-----------------------------------------|----------------------------------------|
| Ниво на виртуализация        | Споделят ядрото на хост ОС              | Всяка VM има собствена ОС              |
| Стартиране                   | Много бързо (милисекунди)               | По-бавно (секунди-минути)              |
| Използване на ресурси        | Леко и ефективно                        | Тежко, по-голям resource overhead      |
| Основна цел                  | Управление на микросървиси и приложения | Управление на инфраструктура и сървъри |
| Основен оркестратор          | Kubernetes, Docker Swarm                | OpenStack, VMware vCenter              |
| Автоматично скалиране        | Вградено и динамично                    | Ограничено или чрез външни инструменти |
| Self-Healing                 | Да – автоматично рестартира контейнери  | Частично – основен failover            |
| Обновления (Rolling Updates) | Автоматизирани без downtime             | По-трудни, често изискват рестарт      |
| Service Discovery            | Вградено в оркестратора                 | Необходимо е външно решение            |
| Типични приложения           | Cloud-native, CI/CD, microservices      | Legacy системи, монолитни приложения   |

# Оркестрация

- Оркестрацията на контейнери е по-гъвкава, автоматизирана и подходяща за модерни cloud-native приложения.
- Оркестрацията на виртуални машини е по-подходяща за традиционни инфраструктури и приложения, изискващи пълна изолация.

# Оркестрация на контейнери

| Инструмент      | Основно предназначение                                   | Кратко описание                                                                     |
|-----------------|----------------------------------------------------------|-------------------------------------------------------------------------------------|
| Kubernetes      | Най-популярна платформа за контейнерна оркестрация       | Автоматизира деплоймънт, скалиране, мрежова свързаност и self-healing на контейнери |
| Docker Swarm    | Вградена оркестрация в Docker                            | Подходящ за по-малки среди, лесен за настройка и интеграция                         |
| HashiCorp Nomad | Универсален оркестратор за контейнери и други приложения | Лек и гъвкав инструмент, може да управлява и non-container workloads                |
| OpenShift       | Enterprise платформа, базирана на Kubernetes             | Осигурява CI/CD, security, monitoring и developer-friendly среди                    |
| Rancher         | Платформа за управление на Kubernetes клъстери           | Централизирано управление на множество Kubernetes инсталации                        |

# Оркестрация на виртуални машини (VMs)

| Инструмент               | Основно предназначение                       | Кратко описание                                                                   |
|--------------------------|----------------------------------------------|-----------------------------------------------------------------------------------|
| OpenStack                | Open-source cloud платформа за виртуализация | Управлява виртуални машини, мрежи и storage ресурси в частен облак                |
| VMware vSphere / vCenter | Комерсиално решение за виртуализация         | Централизирано управление, автоматизация и баланс на натоварването на VMs         |
| Proxmox VE               | Open-source платформа за VMs и контейнери    | Комбинираща KVM виртуализация и LXC контейнери, подходяща за малки и средни среди |
| oVirt                    | Open-source алтернатива на VMware            | Управлява виртуални машини, storage и мрежи                                       |

# Оркестрация в Cloud среди

| Платформа / Инструмент                 | Доставчик               | Основно предназначение            | Кратко описание                                                                               |
|----------------------------------------|-------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------|
| AWS Elastic Kubernetes Service (EKS)   | Amazon Web Services     | Контейнерна оркестрация           | Управлява Kubernetes клъстери в облака с автоматично скалиране и интеграция с AWS услуги      |
| AWS CloudFormation                     | Amazon Web Services     | Инфраструктурна оркестрация (IaC) | Автоматизира създаването на облачни ресурси чрез шаблони (EC2, VPC, IAM и др.)                |
| Google Kubernetes Engine (GKE)         | Google Cloud Platform   | Контейнерна оркестрация           | Управлява Kubernetes клъстери с вградена сигурност и автоматичен scaling                      |
| Google Cloud Deployment Manager        | Google Cloud Platform   | Оркестрация на инфраструктура     | Дефинира и автоматично създава cloud ресурси чрез YAML шаблони                                |
| Azure Kubernetes Service (AKS)         | Microsoft Azure         | Контейнерна оркестрация           | Управлява и скалира Kubernetes клъстери в Azure среда                                         |
| Azure Resource Manager (ARM Templates) | Microsoft Azure         | Инфраструктурна оркестрация       | Управлява cloud ресурси чрез декларативни шаблони                                             |
| Terraform                              | HashiCorp (мулти-cloud) | Универсална оркестрация (IaC)     | Автоматизира и управлява ресурси във всички облачни платформи – AWS, GCP, Azure, VMware и др. |

# Видове оркестрация и основни примери

| Тип оркестрация                 | Основна цел                                             | Ниво на управление                       | Примери за инструменти                                                          | Типични приложения                                    |
|---------------------------------|---------------------------------------------------------|------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------|
| Контейнерна оркестрация         | Автоматизация и управление на контейнери и микросървиси | Приложно ниво                            | Kubernetes, Docker Swarm, OpenShift, Nomad, Rancher                             | Микросървиси, cloud-native приложения, CI/CD процеси  |
| Оркестрация на виртуални машини | Управление на виртуализирана инфраструктура             | Инфраструктурно ниво                     | OpenStack, VMware vCenter, Proxmox VE, oVirt, SCVMM                             | Традиционни приложения, legacy системи, частни облаци |
| Cloud оркестрация               | Автоматизация на инфраструктура и услуги в облака       | Много нива (инфраструктура + приложения) | AWS CloudFormation, GCP Deployment Manager, Azure ARM, Terraform, EKS, GKE, AKS | Облачни платформи, multi-cloud и hybrid среди         |

# Docker Swarm

# Docker Swarm – Вградена оркестрация в Docker

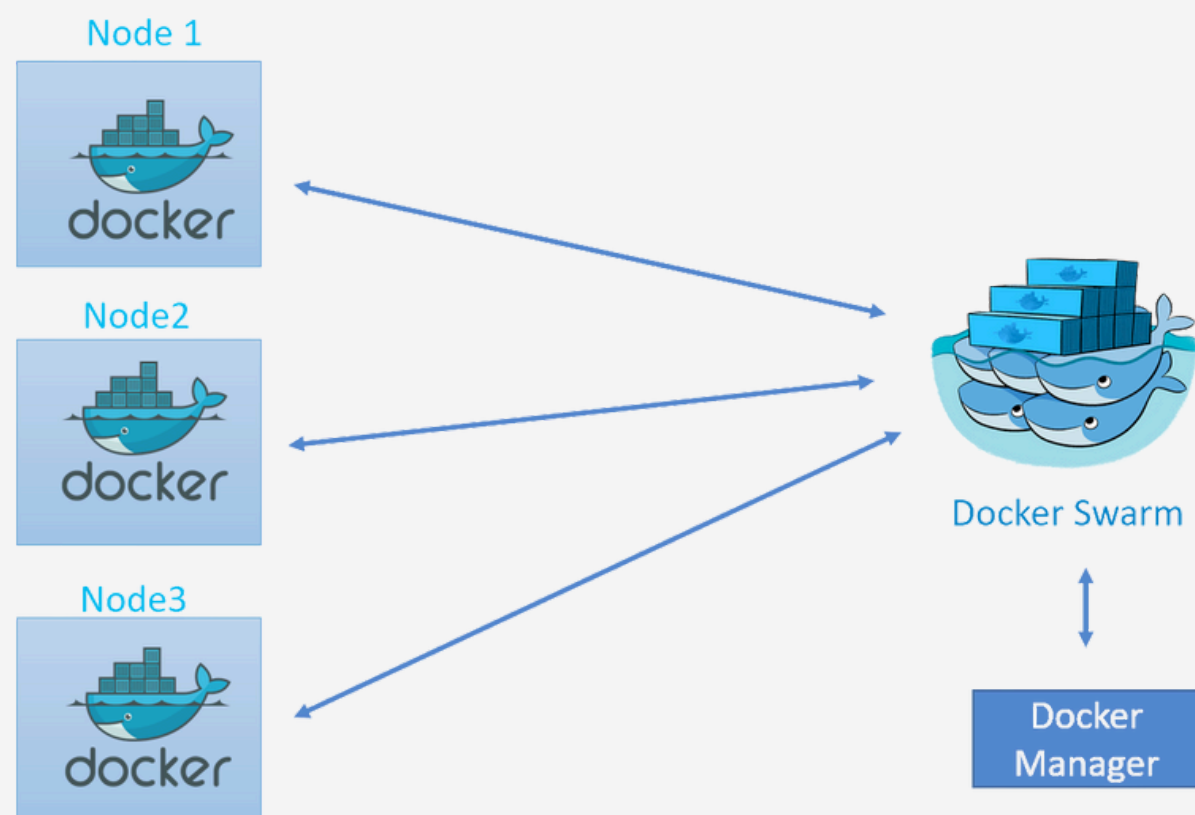
- Инструмент за управление и оркестрация на контейнери в Docker.
- Обединява няколко Docker хоста в един клъстер (Swarm).
- Позволява централизирано управление, скалиране и балансиране на натоварването.
- Лесен за настройка и напълно интегриран с Docker CLI.

# Ключови възможности на Docker Swarm

- Service Discovery – автоматично откриване на контейнери чрез DNS
- Load Balancing – разпределение на трафика между репликите
- Scaling – лесно увеличаване/намаляване на броя контейнери
- Rolling Updates – обновления без спиране на услугата
- High Availability – автоматичен failover между manager nodes

# Архитектура: Manager и Worker nodes

- Manager Nodes – контролират състоянието на кластера, разпределят задачи
- Worker Nodes – изпълняват контейнерите (services)



# Docker Swarm – плюсове и минуси

| Предимства                       | Недостатъци                                    |
|----------------------------------|------------------------------------------------|
| Лесен за инсталация и използване | По-ограничена функционалност от Kubernetes     |
| Интегриран в Docker              | По-слаба community поддръжка                   |
| Подходящ за малки среди          | Не е оптимален за големи production системи    |
| Поддържа rolling updates         | По-малко инструменти за мониторинг и скалиране |

# Docker Swarm в гейстVue

```
# Инициализация на swarm клъстер
docker swarm init

# Добавяне на нов worker node
docker swarm join --token <token> <manager_ip>:2377

# Създаване на service с 3 реплики
docker service create --name webapp --replicas 3 nginx

# Създаване на token за присъединяване на мениджър
docker swarm join-token manager
```

# Ключови възможности на Docker Swarm

- Docker Swarm е лесен и интуитивен инструмент за оркестрация на контейнери.
- Подходящ е за обучения, тестови и малки production среди.
- За по-големи и сложни системи обикновено се предпочита Kubernetes.

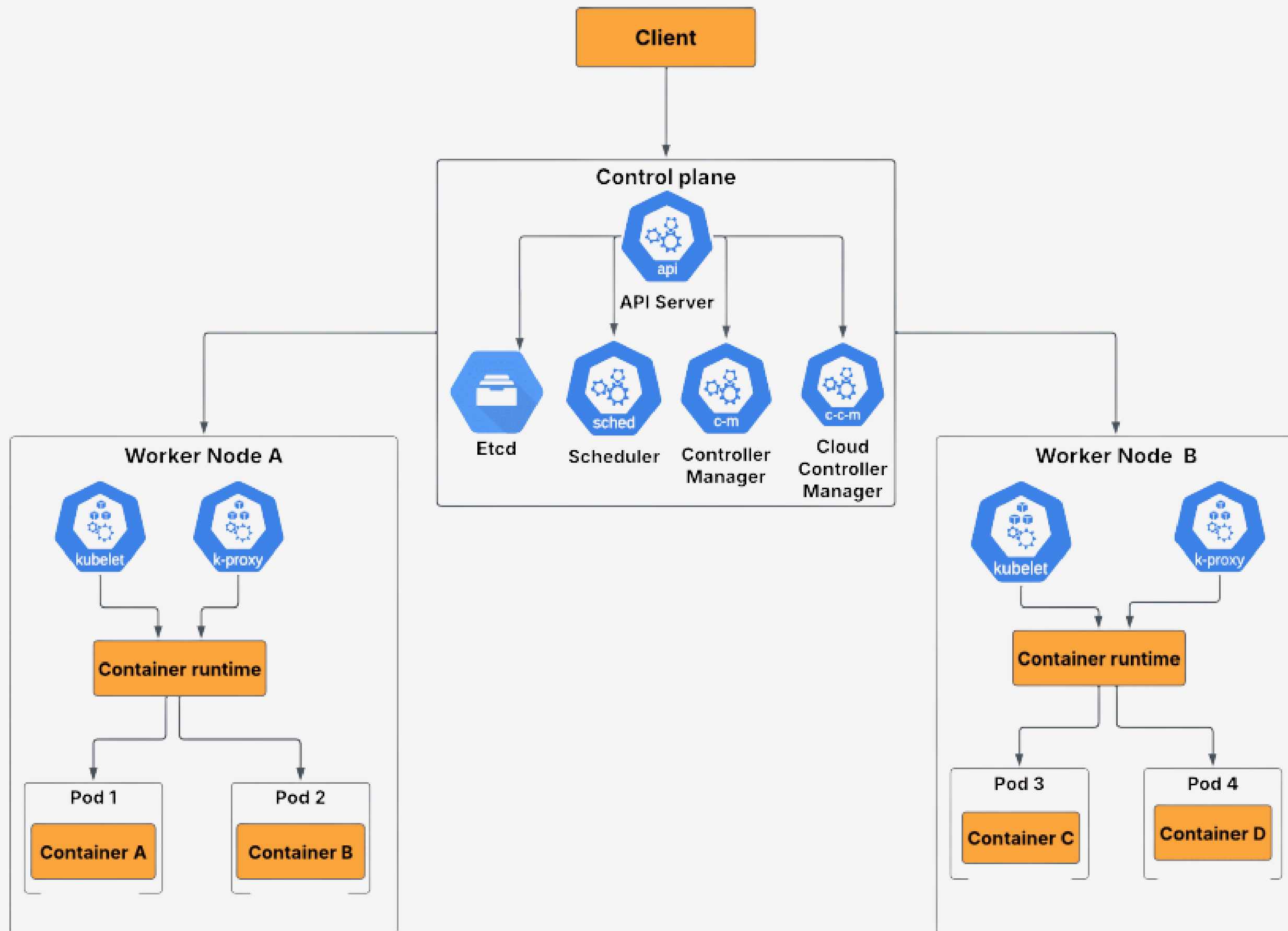
# Kubernetes

# Kubernetes – стандартът за оркестрация на контейнери

- Kubernetes (K8s) е open-source система за оркестрация на контейнери, първоначално разработена от Google.
- Автоматизира деплоймънт, скалиране, управление и self-healing на приложения в контейнери.
- Използва се като дефакто стандарт в cloud и enterprise среди.
- Работи с различни runtime-и (Docker, containerd, CRI-O и др.).

# Ключови възможности на Kubernetes

- Автоматично скалиране според натоварването
- Service Discovery и Load Balancing чрез DNS и вграден прокси
- Rolling Updates и Rollback без прекъсване на услугата
- Self-Healing – автоматично рестартира или пресъздава паднали контейнери
- Secrets и Config Management – сигурно управление на конфигурации
- Multi-Host Networking – обединява контейнери от различни сървъри



# Компоненти и архитектура

| Компонент                | Роля                                                                          |
|--------------------------|-------------------------------------------------------------------------------|
| kube-apiserver           | Приема команди и заявки (комуникация с администраторите и потребителите)      |
| etcd                     | Съхранява цялото състояние на клъстера (като база данни за конфигурацията)    |
| kube-scheduler           | Решава на кой worker node да стартира нов Pod                                 |
| controller-manager       | Следи състоянието и предприема действия (ако Pod падне → стартира нов и т.н.) |
| cloud-controller-manager | Интегрира се с облачни услуги (AWS, GCP, Azure и др.)                         |

# Компоненти и архитектура

- Master (Control Plane): управлява клъстера и взема решения за деплоймънт и скалиране.
- Worker Nodes: изпълняват контейнерите (Pods).
- Pod: най-малката единица в Kubernetes – съдържа един или повече контейнера.

# Kubernetes – плюсове и минуси

| Предимства                            | Недостатъци                                |
|---------------------------------------|--------------------------------------------|
| Изключително мащабируем и стабилен    | По-сложна конфигурация и крива на обучение |
| Автоматизация на почти всички процеси | Изисква повече ресурси за работа           |
| Поддържа multi-cloud и hybrid среди   | Първоначалната настройка е по-трудна       |
| Огромна community поддръжка           | По-труден за управление без DevOps опит    |

# Kubernetes в гейстVue

```
# Създаване на deployment
kubectl create deployment webapp --image=nginx

# Създаване на service (достъп отвън)
kubectl expose deployment webapp --port=80 --type=LoadBalancer

# Скалиране на pods
kubectl scale deployment webapp --replicas=5

# Rolling update
kubectl set image deployment/webapp nginx=nginx:latest

# Проверка на състоянието
kubectl get pods
```

# Обобщение

- Kubernetes е най-мощната и популярна система за оркестрация на контейнери.
- Подходяща за големи, динамични и cloud-native среди.
- Предлага висока автоматизация, надеждност и гъвкавост.
- Днес е индустриален стандарт за управление на контейнери в production среди.